Bottom-Up Parsing

26 March, 2026

Bottom-up parsing

Our objective is to construct a parse tree for an input string beginning at the leaves (*bottom*) and working *up* towards the root.
At each reduction step, a particular substring matching the right side of a production is replaced by the symbol on the left of the production and if the substring is chosen correctly at each step, a right-most derivation is traced out in reverse.
For example, consider a grammar with the following production rules:

$$S \to aABc$$
$$A \to Abc \mid b$$
$$B \to d$$

, The bottom-up parsing for the input string $abbcde$ is

$$abbcdc \Rightarrow aAbcdc \Rightarrow aAdc \Rightarrow aABc \Rightarrow S$$

Handle

A handle of a right sentential form γ is a production $A \to \beta$ and a position in γ where the string β may be found and replaced by A to produce the previous right sentential form in a right-most derivation of γ.
For example, consider the following sentential forms:

$$S \overset{*}{\Rightarrow} \alpha Aw \Rightarrow \alpha\beta w$$

then $A \to \beta$ and the position following α is a handle of $\alpha\beta w$.
In the previous example:

$$abbcdc \Rightarrow aAbcdc$$

a handle of $abbcdc$ is the production $A \to b$ at position 2.
Likewise, in the case of:

$$aAbcdc \Rightarrow aAdc$$

a handle of $aAbcdc$ is the production $A \to Abc$ at position 2.

Handle pruning

We start with the string of terminals w that we wish to parse.
If w is a sentence of the grammar at hand, then $w = \gamma_n$ where γ_n is the n^{th} right sentential form of some as yet unknown right-most derivation.

$$S = \gamma_0 \Rightarrow \gamma_1 \Rightarrow \gamma_2 ... \Rightarrow \gamma_n = w$$

To reconstruct this derivation in reverse order, we locate the handle β_n in γ_n and replace β_n with the left side of some production $A_n \to \beta_n$ to obtain the $n-1^{\text{th}}$ right sentential form γ_{n-1}.
We then repeat this procedure and if by continuing this we produce a right sentential form consisting only of the start symbol S, then we halt and announce successful completion of parsing. The reverse of the sequence of productions used in the reduction is the right-most derivation of the input string.
For example, consider a grammar with the following production rules:

$$E \to E + E$$
$$E \to E * E$$
$$E \to (E)$$
$$E \to \text{id}$$

and the input string: $\text{id}_1 + \text{id}_2 * \text{id}_3$

Right sentential form	Handle	Reduction production
$\text{id}_1 + \text{id}_2 * \text{id}_3$	id_1	$E \to \text{id}$
$E + \text{id}_2 * \text{id}_3$	id_2	$E \to \text{id}$
$E + E * \text{id}_3$	id_3	$E \to \text{id}$
$E + E * E$	$E * E$	$E \to E * E$
$E + E$	$E + E$	$E \to E + E$
E	-	-

Table 1: Reductions made by shift-reduce parser

Stack implementation of shift-reducing parser

A convenient way to implement a shift-reduce parser is to use a stack to hold grammar symbols and an input buffer to hold the string w to be parsed (we use \$ to mark the top of the stack and the right-end marker).

	Stack	Input
Initially stack is empty and w is in the input	\$	w\$
Finally stack contains the start symbol	\$$S$	\$

Table 2: State of shift-reduce parser

There are four possible actions a shift-reduce parser can make:
1. *Shift*: Shift the next input symbol onto the top of the stack
2. *Reduce*: The right end of the string to be reduced must be at the top of the stack. Locate the left end of the string within the stack and decide with what non-terminal to replace the string.
3. *Accept*: Announce successful completion of the parsing.
4. *Error*: Discover a syntax error and call an error recovery routine.

Stack	Input	Action
$\$$	$\mathrm{id}_1 + \mathrm{id}_2 * \mathrm{id}_3\$$	*Shift*
$\$\mathrm{id}_1$	$+ \mathrm{id}_2 * \mathrm{id}_3\$$	*Reduce* by $E \to \mathrm{id}$
$\$E$	$+ \mathrm{id}_2 * \mathrm{id}_3\$$	*Shift*
$\$E +$	$\mathrm{id}_2 * \mathrm{id}_3\$$	*Shift*
$\$E + \mathrm{id}_2$	$* \mathrm{id}_3\$$	*Reduce* by $E \to \mathrm{id}$
$\$E + E$	$* \mathrm{id}_3\$$	*Shift*
$\$E + E *$	$\mathrm{id}_3\$$	*Shift*
$\$E + E * \mathrm{id}_3$	$\$$	*Reduce* by $E \to \mathrm{id}$
$\$E + E * E$	$\$$	*Reduce* by $E \to E * E$
$\$E + E$	$\$$	*Reduce* by $E \to E + E$
$\$E$	$\$$	*Accept*

Table 3: Shift-reduce table

LR(k) parsing

It is an efficient bottom-up parsing technique that can be used to parse a large class of context-free grammars.
The L is for left-right scanning of input.
The R is for constructing the right-most derivation in reverse.
k is for the number of input symbols of lookahead that are used in making parsing decisions. When (k) is omitted, it is assumed to be 1.

LR parser

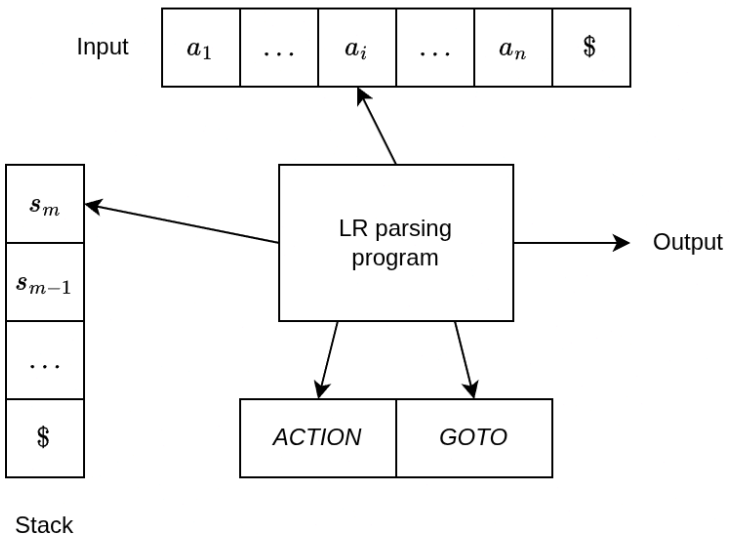

Figure 1: Structure of LR parser

The LR parser consists of an input, an output, a stack, a driver program and a parsing table which has two parts: *ACTION* and *GOTO*

LR parsing algorithm

Input: An input string w and an LR-parsing table with functions ACTION and GOTO for a grammar G.
Output: If w is in $L(G)$, the reduction steps of a bottom-up parse for w, otherwise an error.

Method: Initially, the parser has s_0 on its stack, where s_0 is the initial state, and $w\$$ in the input buffer.

let a be the first symbol of $w\$$;
while (1) {
  let s be the state on top of the stack;
  if (ACTION$[s, a]$ = Shift t) {
    push a and t onto the stack;
  } else if (ACTION$[s, a]$ = Reduce $A \to \beta$) {
    pop $2|\beta|$ symbols off the stack;
    let state t now be on top of the stack;
    push A and GOTO$[t, A]$ onto the stack;
    output the production $A \to \beta$;
  } else if (ACTION$[s, a]$ = Accept) break;
  else call error-recovery routine;
}

Example 1

A grammar G contains the following production rules:

$$E \to E + T \qquad E \to T$$
$$T \to T * F \qquad T \to F$$
$$F \to (E) \qquad F \to \text{id}$$

State	Action						Goto		
	id	+	*	(	)	$	E	T	F
0	s_5			s_4			1	2	3
1		s_6				Accept			
2		r_2	s_7		r_2	r_2			
3		r_4	r_4		r_4	r_4			
4	s_5			s_4			8	2	3
5		r_6	r_6		r_6	r_6			
6	s_5			s_4			9	3	
7	s_5			s_4					10
8		s_6			s_{11}				
9		r_1	s_7		r_1	r_1			
10		r_3	r_3		r_3	r_3			
11		r_5	r_5		r_5	r_5			

Table 4: Parsing table for G

	STACK	INPUT	ACTION
1	0	$\text{id} + \text{id} * \text{id}$ \$	Shift
2	0 id 5	$+ \text{id} * \text{id}$ \$	Reduce by $F \to \text{id}$
3	0 F 3	$+ \text{id} * \text{id}$ \$	Reduce by $T \to F$
4	0 T 2	$+ \text{id} * \text{id}$ \$	Reduce by $E \to T$
5	0 E 1	$+ \text{id} * \text{id}$ \$	Shift
6	0 E 1 $+$ 6	$\text{id} * \text{id}$ \$	Shift
7	0 E 1 $+$ 6 id 5	$* \text{id}$ \$	Reduce by $F \to \text{id}$
8	0 E 1 $+$ 6 F 3	$* \text{id}$ \$	Reduce by $T \to F$
9	0 E 1 $+$ 6 T 9	$* \text{id}$ \$	Shift
10	0 E 1 $+$ 6 T 9 $*$ 7	id \$	Shift
11	0 E 1 $+$ 6 T 9 $*$ 7 id 5	\$	Reduce by $F \to \text{id}$
12	0 E 1 $+$ 6 T 9 $*$ 7 F 10	\$	Reduce by $T \to T * F$
13	0 E 1 $+$ 6 T 9	\$	Reduce by $E \to E + T$
14	0 E 1	\$	Accept

Table 5: Moves of an LR parser on $\text{id} + \text{id} * \text{id}$